

Buổi 3: Thực hành Git theo nhóm (3 sinh viên/nhóm)

Mục tiêu buổi học

- Làm quen với quy trình làm việc nhóm với Git.
 - Thành thạo tạo branch, commit, push, merge, xử lý xung đột.
 - Thực hành tạo PR/MR, tag, release, rollback.
 - Nâng cao kỹ năng giao tiếp và phối hợp qua Git.
-

Hoạt động 1: Chuẩn bị môi trường nhóm

Yêu cầu thực hành:

1. Sinh viên tạo repository trung tâm `GroupProject-Buoi3`.
2. Mỗi nhóm clone repo về máy cá nhân (3 thành viên, mỗi bạn thực hiện).
3. Mỗi bạn tạo branch cá nhân: `feature-[tên]`.
4. Push branch cá nhân lên remote.
5. Lập lại: mỗi bạn tạo thêm 1 branch thử nghiệm `test-[tên]` và push.

Sản phẩm nộp:

- URL repo trung tâm.
 - Screenshot hiển thị ≥ 2 branch của mỗi thành viên.
 - **Giải thích:** Vì sao cần branch cá nhân khi làm việc nhóm?
-

Hoạt động 2: Tạo file và commit trên branch cá nhân

Yêu cầu thực hành:

1. Mỗi thành viên tạo 2 file: `task-[tên].md`, `note-[tên].txt`.
2. Commit lần 1 với thông điệp chuẩn.
3. Commit lần 2 sau khi chỉnh sửa.
4. Push lên branch cá nhân.
5. Lập lại: thêm 1 file mới, commit thêm lần nữa.

Sản phẩm nộp:

- Screenshot `git log --oneline --graph` ≥ 3 commit/branch.
 - **Giải thích:** Vì sao cần commit nhiều lần thay vì gộp chung?
-

Hoạt động 3: Tạo Pull Request (PR) / Merge Request (MR)

Yêu cầu thực hành:

1. Mỗi thành viên tạo PR/MR để merge branch cá nhân vào `develop`.
2. Thành viên khác review PR (ít nhất 1 comment).
3. Nhóm trưởng merge sau khi review.
4. Lặp lại: mỗi thành viên làm thêm 1 PR nhỏ.

Sản phẩm nộp:

- Screenshot PR/MR đã merge.
 - Screenshot comment review.
 - **Giải thích:** Vì sao cần review code trước khi merge?
-

Hoạt động 4: Thực hành xung đột merge

Yêu cầu thực hành:

1. Thành viên A và B cùng sửa cùng một dòng trong `README.md`.
2. Commits và push.
3. Nhóm trưởng merge → gây conflict.
4. Giải quyết conflict thủ công, commit lại.
5. Lặp lại: thực hiện lại với file `task-shared.md`.

Sản phẩm nộp:

- Screenshot file chứa conflict markers.
 - Screenshot file sau khi sửa xong.
 - **Giải thích:** Vì sao conflict xảy ra? Làm sao hạn chế conflict?
-

Hoạt động 5: Quản lý tag và release

Yêu cầu thực hành:

1. Nhóm trưởng tạo tag `v1.0` cho commit mới nhất.
2. Push tag lên remote.
3. Mỗi thành viên clone repo mới, checkout `v1.0`.
4. Lặp lại: Nhóm trưởng chỉnh sửa nhỏ → tạo `v1.1`, push. Mọi người checkout lại.

Sản phẩm nộp:

- Screenshot `git tag` hiển thị cả `v1.0` và `v1.1`.
- Screenshot checkout repo tại `v1.0`, `v1.1`.
- **Giải thích:** Khác biệt giữa branch và tag?

Hoạt động 6: Báo cáo nhóm

Yêu cầu thực hành:

- Nhóm viết báo cáo (2–3 trang):
 - Quy trình làm việc nhóm.
 - Các bước Git đã thực hiện.
 - Vấn đề & cách khắc phục.
 - Cảm nhận cá nhân.
- Lập lại: mỗi thành viên viết 1 phần cảm nhận riêng.

Sản phẩm nộp:

- File báo cáo PDF.
 - Giải thích:** Nhóm học được gì khi so sánh làm việc nhóm có Git và không có Git?
-

Hoạt động 7: Làm việc với branch feature chung

Yêu cầu thực hành:

- Nhóm trưởng tạo branch `feature-shared`.
- Mỗi thành viên: checkout vào branch này, thêm file `shared-[tên].txt`, commit và push.
- Lập lại: cả nhóm cùng chỉnh sửa file `group-note.md`, commit, push.
- Nhóm trưởng merge `feature-shared` vào `develop`.

Sản phẩm nộp:

- Screenshot `git log --oneline --graph` thể hiện commit của cả 3 thành viên.
 - Screenshot merge vào `develop`.
 - Giải thích:** So sánh làm việc trên branch riêng và branch chung.
-

Hoạt động 8: Thực hành rollback và phục hồi

Yêu cầu thực hành:

- Một thành viên commit sai nội dung vào `README.md`.
- Thực hiện rollback bằng `git reset --soft HEAD~1`.
- Commit lại nội dung đúng.
- Thành viên khác rollback bằng `git revert`.
- Lập lại: tạo commit lỗi mới, dùng `git restore` để phục hồi file.

Sản phẩm nộp:

- Screenshot log hiển thị thao tác reset, revert.
- Screenshot file sau khi restore.
- **Giải thích:** Khác biệt giữa reset, revert, restore? Nên dùng cách nào trong nhóm?